

Very Busy Expressions

1. Introduzione

Che cos'è una Very Busy Expression

 **Definizione** Un'espressione è **very busy** in un punto `p` se, indipendentemente dal percorso preso da `p`, l'espressione viene usata prima che uno dei suoi operandi venga definito.

Un'espressione `a+b` è **very busy** in un punto `p` se `a+b` è valutata in tutti i percorsi da `p` a `EXIT`.

Eventuali ridefinizioni di `a` o `b` lungo tali percorsi sono presenti solo **dopo** la valutazione

Esempi

Per capire meglio che cos'è una **Very Busy Expression** possiamo fare degli esempi dove una espressione è Very Busy e quando invece non lo è.

1. È una Very Busy Expression

```
entry: p -> inizio programma
BB1:   if (condizione) {
BB2:       x = a + b;
        } else {
BB3:       y = a + b;
        }
```

In `p`, l'espressione `a+b` è Very Busy. Perché, verso `EXIT`, in `BB2` e in `BB3` l'espressione viene valutata.

Così abbiamo un vantaggio, ovvero quello che il compilatore può abilitare il "Code Hoisting" ovvero "solleva" `a+b` prima della condizione `if` calcolandola una sola volta per risparmiare spazio nel codice.

Avremmo quindi un codice più compatto:

```
t1 = a + b;
if (condizione) {
    x = t1;
} else {
```

```
y = t1;  
}
```

Avremmo anche minore spazio occupato, in quanto la somma compare solo una volta.

Anche se la CPU eseguirà comunque una sola somma, il vantaggio è a livello di memoria e cache:

- il file eseguibile diventa più piccolo
- più il codice è compatto, più è probabile che rimanga nella cache della CPU, migliorando le prestazioni generali
- è un modo per eliminare la ridondanza

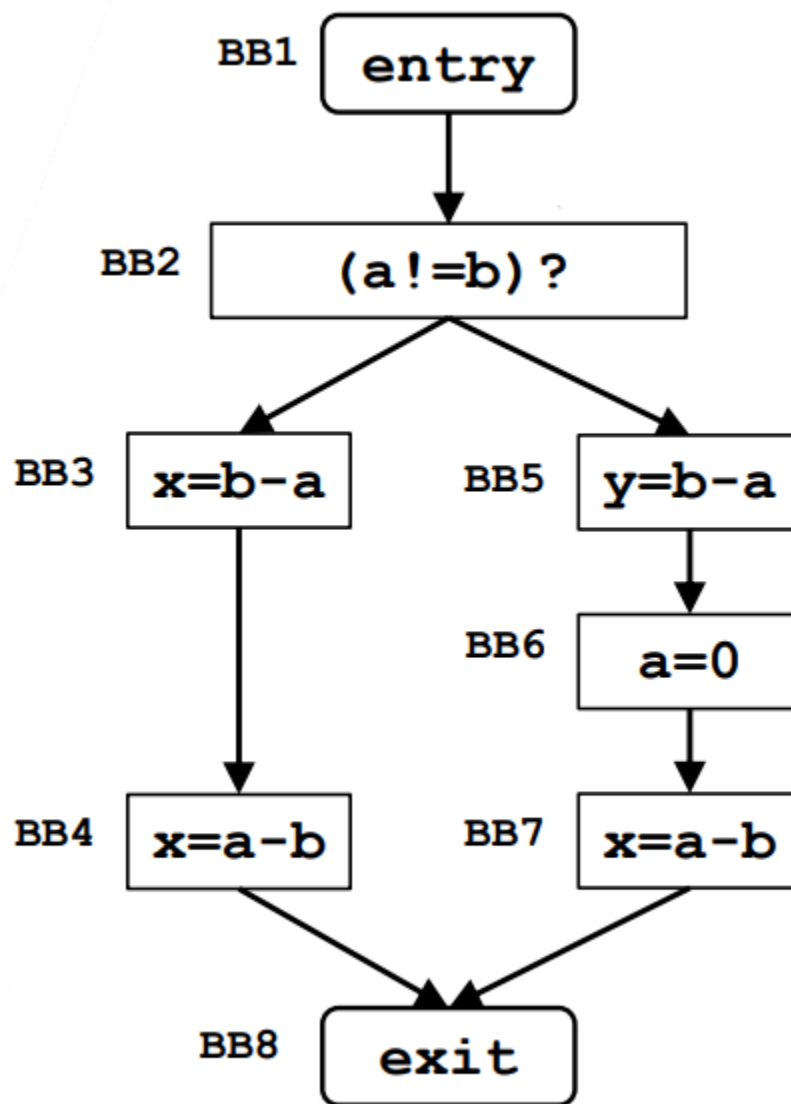
2. NON è una Very Busy Expression

```
entry: p -> inizio programma  
BB1:   if (condizione) {  
BB2:     x = a + b;  
      } else {  
BB3:     a = 10;  
        y = a + b;  
      }
```

⚠ **Attenzione** In `p`, l'espressione `a+b` **NON** è Very Busy. Perché, verso `EXIT`, in `BB3`, `a` viene ridefinito prima della valutazione. il valore di `a+b` in `p` non è lo stesso che verrà calcolato in `BB2`; questo perché `BB3` "uccide" l'espressione `a+b`.

Non possiamo anticipare il calcolo in `p`.

2. Framework



Dopo aver chiarito e fatto esempi su cosa sono le **Very Busy Expressions**, possiamo studiare il nostro **Control Flow Graph (CFG)** fornito.

Definizione del dominio

Il dominio rappresenta l'insieme di tutti i possibili valori che l'analisi può assumere in ogni punto del programma.

Il dominio è composto dall'insieme di tutte le espressioni aritmetiche presenti nel CFG, ovvero le espressioni $b - a$, $a - b$.

In un'implementazione reale, usiamo un bit vector dove ogni bit corrisponde a un'espressione del dominio

- bit 0 : $b - a$

- bit 1 : $a - b$

Ad esempio, se in un punto il bit vector è 10 , significa che solo $b - a$ è Very Busy; se è 11 lo sono entrambe

Direzione

Ora che abbiamo il dominio, possiamo stabilire la direzione.

📌 **Direction: Backward** Poiché dobbiamo verificare cosa accadrà "nel futuro" del programma per decidere se un'espressione è utilizzabile nel presente, la Direction è **Backward** (all'indietro). Partiamo dalla fine (EXIT) e risaliamo verso l'inizio (ENTRY).

Da qui possiamo procedere alla definizione di tutti gli altri parametri del framework.

Meet Operation

Come abbiamo visto, una Very Busy Expression lo è **solo se è presente in tutti i percorsi possibili**. Se anche in un solo ramo l'espressione manca o viene uccisa da una ridefinizione, non possiamo considerarla Busy nel punto precedente. L'operatore **aritmetico** che modello questo concetto di "tutti" è l'**intersezione** ($\wedge$ o $\cap$).

Funzione di trasferimento

La funzione di trasferimento calcola come le informazioni cambiano attraversando un singolo blocco di codice. Nel nostro caso, poiché l'analisi è **Backward**, la funzione ci dice quali espressioni sono "busy" all'inizio (*IN*), basandosi su ciò che accade all'uscita (*OUT*) e all'interno del blocco stesso.

La formula è:

$$IN[B] = Gen[B] \cup (OUT[B] - Kill[B])$$

Analizziamola:

1. $Gen[B]$ rappresenta le **espressioni** che vengono calcolate all'**interno** del blocco prima che i loro operandi vengano eventualmente cambiati. Per esempio, nel BB3 ($x = b - a$), il blocco usa l'espressione $b - a$, quindi $Gen[BB3] = b - a$. Questa informazione viene generata e mandata in alto.
2. $Kill[B]$ rappresenta le espressioni che **non sono più valide** perché uno dei loro operandi (le variabili che le compongono) viene ridefinito all'**interno** del blocco. Per esempio, nel BB6 ($a = 0$), il blocco cambia il valore di a . Tutte le espressioni nel dominio che contengono a (ovvero $b - a$, $a - b$) vengono **uccise**. Quindi $Kill[BB6] = b - a$, $a - b$.

3. $OUT[B] - Kill[B]$ prende le espressioni che arrivano dal basso (OUT) e **rimuove** quelle che il blocco ha reso invalide ($Kill$). Se dal basso arriva l'informazione che $a - b$ è **busy**, ma il blocco corrente fa $a = 0$, l'espressione viene rimossa perché non è più la stessa che avevamo prima.
4. $\cup$, uniamo le espressioni che il blocco stesso ha calcolato (Gen) con quelle che sono riuscite a passare vive attraverso il blocco dal basso ($OUT - Kill$).

Facciamo un esempio pratico del blocco BB6. Ipotizziamo che dal basso (da BB7) arrivi $a - b$:

1. $OUT[BB6] = a - b$ (arriva da BB7)
2. $Kill[BB6] = b - a, a - b$ (perché a viene azzerata)
3. $Gen[BB6] = \emptyset$ (non ci sono calcoli aritmetici)

Applichiamo la formula:

$$IN[BB6] = \emptyset \cup (a - b - b - a, a - b)$$

$$IN[BB6] = \emptyset \cup \emptyset = \emptyset$$

☰ **Conclusione** Anche se $a - b$ era busy dopo il blocco, non lo è più all'inizio del blocco perché l'istruzione $a = 0$ l'ha uccisa.

Boundary condition

Stabilisce il valore fisso per il punto in cui l'analisi entra nel programma.

- poiché l'analisi è **Backward**, il punto di ingresso è la fine del programma, ovvero EXIT.
- Valore $IN[EXIT] = \emptyset$

Immaginiamo di essere alla fine del codice. Non ci sono più istruzioni da eseguire. Di conseguenza, non esiste alcuna espressione che verrà usata in futuro. Il futuro è vuoto, quindi l'insieme delle espressioni Busy è vuoto.

Initial Interior Points

Stabilisce quale valore dare a tutti gli altri blocchi prima che l'algoritmo inizi a calcolare valori reali.

- Valore $IN[B] = \mathbb{U}$ (insieme universale, ovvero tutte le espressioni $b - a, a - b$)
- Poiché la Meet Operation è l'intersezione $\cap$:
 - Se inizializzassimo a $\emptyset$ (vuoto), l'intersezione tra $\emptyset$ e qualunque altra cosa darebbe sempre $\emptyset$. L'analisi si bloccherebbe subito a "nessuna espressione è busy", fallendo nel trovare le ottimizzazioni.

- Iniziando con $\mathbb{U}$, diciamo al compilatore: "Assumi che tutto sia *Very Busy*, finché non ti dimostro il contrario". Durante le iterazioni, l'intersezione "taglierà via" le espressioni che non sono veramente busy in tutti i rami, scendendo verso il valore corretto.

Tabella del Framework

Parametro	Valore per Very Busy Expressions
Domain	Insieme delle espressioni $b - a, a - b$
Direction	Backward
Transfer function	$IN[B] = Gen[B] \cup (OUT[B] - Kill[B])$
Meet Operation ($\wedge$)	Intersezione ($\cap$)
Boundary Condition	$IN[EXIT] = \emptyset$
Initial interior points	$IN[B] = \mathbb{U}$

3. Tabella delle iterazioni

Legenda: $e_1 = (b - a)$, $e_2 = (a - b)$.

Blocco	Inizializzazione IN[B]	Iterazione 1 OUT[B]	Iterazione 1 IN[B]	Iterazione 2 OUT[B]	Iterazione 2 IN[B]
BB8 (EXIT)	$\emptyset$	-	$\emptyset$	-	$\emptyset$
BB7	e_1, e_2	$\emptyset$	e_2	$\emptyset$	e_2
BB6	e_1, e_2	e_2	$\emptyset$	e_2	$\emptyset$
BB5	e_1, e_2	$\emptyset$	e_1	$\emptyset$	e_1
BB4	e_1, e_2	$\emptyset$	e_2	$\emptyset$	e_2
BB3	e_1, e_2	e_2	e_1, e_2	e_2	e_1, e_2
BB2	e_1, e_2	e_1	e_1	e_1	e_1
BB1	e_1, e_2	e_1	e_1	e_1	e_1

L'algoritmo termina all'**Iterazione 2** perché i valori dei set *IN* e *OUT* sono identici a quelli dell'Iterazione 1.

È fondamentale notare che $OUT[BB6] = e_2$ (ricevuto da BB7), ma l'istruzione `a = 0` genera un $Kill[BB6] = e_1, e_2$, rendendo $IN[BB6] = \emptyset$.

L'operazione di **Meet (Intersezione)** tra $IN[BB3] = e_1, e_2$ e $IN[BB5] = e_1$ produce e_1 , poiché solo $b - a$ è comune a entrambi i rami.

✓ **Risultato finale** Poiché $IN[BB1] = e_1$, in questo specifico CFG è possibile effettuare il *code hoisting* dell'espressione $b - a$ prima della condizione `(a != b)`.